

Design Document

Introduction:

This document describes the design and implementation of a Blockchain-enabled Flight Travel Insurance System built on the Ethereum platform. The goal of the project is to demonstrate how smart contracts can automate insurance policy management and claim payouts without requiring a centralized authority. Inspired by real-world projects from AXA and Etherisc, the system allows passengers to purchase flight insurance and automatically receive compensation when extreme weather affects their departure city.

System Overview:

Passengers purchase flight insurance policies and automatically receive indemnity when extreme weather such as Hail or Flood occurs in their departure city on their flight date. The system uses a Solidity smart contract deployed on a local Hardhat Ethereum node, with JavaScript scripts handling deployment, policy purchases, and claim processing.

Roles:

There are two roles: the Insurance Provider, which is the account that deploys the contract and is the only one allowed to view all policies and pay indemnities, and the Passenger, which is any other account that can purchase a policy and check their policy status and balance.

Project Structure:

The project contains FlightInsurance.sol for the smart contract, and four scripts: deploy.js to deploy the contract, purchase_policies.js and purchase_policies2.js for two rounds of policy purchases, and verify_and_pay.js to process claims using weather.txt as the external data source.

Smart Contract Design:

The contract defines a Policy struct storing the passenger name, address, flight number, date, departure city, destination city, and status. Policies are stored in a mapping keyed by passenger address. An onlyProvider modifier restricts sensitive functions to the insurance provider. The premium is fixed at 0.01 Ether and the indemnity at 0.02 Ether as contract constants.

Phase I Functions:

view_available_policy returns the policy terms as a string. purchase_policy creates a policy and transfers the 0.01 Ether premium to the provider. view_purchased_policy lets a passenger check their own policy using their address as the key. view_all_policies is provider-only and returns all purchased policies.

Phase II Functions:

pay_indemnity is called by the provider to send 0.02 Ether to a passenger and update their status to claimed. view_balance returns the caller's current ETH balance. The verify_and_pay.js script reads weather.txt, fetches all policies from the blockchain, and calls pay_indemnity for any passenger whose departure city had Hail or Flood on their flight date.

ETH Flow:

Passengers send 0.01 Ether to the provider when purchasing a policy. If a claim is verified, the provider sends 0.02 Ether back to the passenger, resulting in a net gain of 0.01 Ether for the passenger.

Tools and Technologies:

The contract is written in Solidity 0.8.28. Hardhat provides the local Ethereum node with 20 funded test accounts. ethers.js is used in the JavaScript scripts to interact with the blockchain. Node.js is the runtime for all scripts.

Test Cases:

In round one, Alice (Phoenix/Flood) and Bob (Denver/Hail) were paid 0.02 Ether each, while Carol (Austin/Normal) received no payout. In round two, Dave (Tampa/Hail) and Eve (Miami/Flood) were paid, while Frank (Houston/Rainfall) and Grace (Tucson/Normal) were not.

Measurements:

Deploying the contract cost approximately 800,000 gas, purchase_policy cost approximately 150,000 gas, and pay_indemnity cost approximately 50,000 gas. Read-only functions like view_all_policies cost zero gas since they do not modify blockchain state.

Conclusion:

This project successfully demonstrates how Ethereum smart contracts can be used to automate a flight insurance system. Phase I established the core policy management functionality, allowing passengers to purchase policies and the provider to manage them securely on the blockchain. Phase II extended the system to process real weather data and automatically pay indemnities to affected passengers without manual intervention. The system is transparent, tamper-proof, and eliminates the need for a centralized insurance authority, showcasing the practical value of blockchain technology in real-world applications.